Trabalho Prático II
Relatório

Aluno: Jean Seixas de Souza

Matrícula: 22154600

1. Decisões de projeto

1.1) Layout do registro em disco:

Optei por um layout de tamanho fixo com padding, totalizando 1.522 bytes por registro.

Campo	Bytes	Descrição
id	4	Inteiro
ano	4	Inteiro
citações	4	Inteiro
len_titulo	4	Comprimento real do título
len_autores	4	Comprimento real da lista de autores
len_atualiz	4	Comprimento real da data/hora
len_snippet	4	Comprimento real do resumo
título	300	Máximo do enunciado
autores	150	Máximo do enunciado
atualização	20	AAAA-MM-DD HH:MM:SS
snippet	1024	Máximo do enunciado
Tota Total	**1522**	

O tamanho fixo permite acesso aleatório O(1) dentro do bloco, sem manter um diretório de offsets. O desperdício de espaço é compensado pela simplicidade de leitura e pelo fato de que o bloco de dados comporta múltiplos registros mesmo assim.

Truncamento: Títulos com mais de 300 bytes, autores com mais de 150, etc. São truncados silenciosamente. Isso não ocorreu na amostra oficial.

1.2) Cabeçalho de bloco de dados:

Cada bloco tem um cabeçalho de 16 bytes.

Número de slots por bloco:

- 4.096 B: ‘(4096 − 16) / 1522 = 2’ slots
- 8.192 B: ‘(8192 − 16) / 1522 = 5’ slots

1.3) Hashing

Função: Mistura multiplicativa estilo ‘splitmix’, escolhida por distribuir bem IDs sequênciais sem exigir análise prévia de distribuição.

Buckets: 2.048. Com 204.285 registros, a média é de 100 registros por bucket. Como cada bloco comporta apenas 2 registros (4.096 B), espera-se overflow generalizado.
Colisão/overflow: Lista encadeada. Cada bucket tem um bloco principal; quando ele enche, um novo bloco é alocado por ‘alloc_block()’ e o campo ‘proximo_bloco_overflow’ do bloco anterior é atualizado.
Bucket 0 do arquivo é o ‘header’ global: ‘[num_buckets][total_registros]’.

1.4) B+Tree

Ordem: Calculada em tempo de compilação.

4.096	B	→	‘338’	chaves	por	nó
8.192	B	→	‘680’	chaves	por	nó

Estrutura do nó: 20 bytes de cabeçalho (is_leaf, num_keys, parent, next_leaf, reserved), seguidos de um vetor de chaves int32 e um vetor de filhos uint64.

Chave = int32
Índice primário: o próprio id.
Índice secundário: os 4 primeiros bytes do título, interpretados como int32. É um fingerprint compacto, não um hash completo.

Valor = uint64 = offset absoluto do registro no arquivo de dados (block_idx * BLOCK_SIZE + DATA_HEADER_SIZE + slot * RECORD_SIZE + 4).

Chaves duplicadas (índice secundário): Inseridas na mesma folha em sequência; quando o nó transborda, é feito split normal. A consulta percorre next_leaf para coletar todas.

Folhas encadeadas: Habilitam seekrange em O(log n + k).

2. Previsão antes da medição

2.1 Ordem e altura da B+Tree

Para 204.285 registros:

BLOCK_SIZE	Ordem	Folhas necessárias	Altura esperada
4.096	338	`⌈204285/338⌉ ≈ 605`	3 (raiz + 1 interno + folhas)
8.192	680	`⌈204285/680⌉ ≈ 301`	2(raiz + folhas)

2.2 Custo esperado por operação

Operação	4.096 esperado	8.192 esperado
findrec(hash)	1 bloco se sem overflow; até ~50 com overflow	~1 a ~25
seek1 (B+Tree ID)	3 (descida) + 1 (dado) = 4	2 + 1 = 3
findtitle	varredura completa ≈ 102.660	≈ 41.682
seek2(B+Tree título)	3 + k	2 + k
findrange	varredura completa ≈ 102.660	≈ 41.682
seekrange (B+Tree ID)	3 + folhas + k	2 + folhas + k

2.3 Previsões de comportamento

findrec: Mais barato que seek1 para IDs em buckets sem overflow.
findtitle: Muito mais caro que seek2 (~25.000× para título único).
findrange: Muito mais caro que seekrange para intervalos pequenos.
Overflows no hash: Esperados em quase todos os buckets.

3.Medição

3.1 Totais dos arquivos

Arquivo	4.096 B	8.192 B
artigos.dat	102.660	41.682
idx_id.btree	1.217	602

idx_titulo.btree	997	487
Ordem B+Tree	338	680
Altura B+Tree (ID)	3	2 680
Altura B+Tree (Tít.)	3	2
Registros por bloco	2	5
Taxa de ocupação	99,50 %	98,02 %
Linhas ignoradas	2	2

Os totais foram impressos pelo programa upload ao fim da carga e provêm diretamente das constantes calculadas a partir de blockio::kBlockSize. Com 4.096 bytes, o cabeçalho de bloco de 16 bytes e o slot de registro de 1.522 bytes, deixam espaço para apenas dois registros por bloco — o que é confirmado pela linha Registros por bloco: 2 no log. Com 8.192 bytes, o mesmo cálculo passa a caber cinco registros por bloco, razão pela qual o arquivo de dados encolhe de 102.660 para 41.682 blocos. A altura da B+Tree cai de 3 para 2 porque a BTREE_MAX_KEYS cresce de 338 para 680: as ~301 folhas necessárias em 8.192 B cabem sob uma única raiz de primeiro nível, enquanto em 4.096 B são necessárias ~605 folhas, o que força um nível interno adicional. As duas linhas ignoradas são as mesmas em ambos os experimentos porque o leitor CsvReader descarta linhas malformadas independentemente do tamanho de bloco.

3.2 IDs existentes

ID	findrec 4096	findrec 8192	seek1 4096	seek1 8192
5	1	1	5 + 1 = 6	4 + 1 = 5
316639	11	5	5 + 1 = 6	4 + 1 = 5
663752	25	10	5 + 1 = 6	4 + 1 = 5
1014283	35	14	5 + 1 = 6	4 + 1 = 5
1483294	50	20	5 + 1 = 6	4 + 1 = 5

Cada valor em findrec é o número de iterações do “while (block_idx != NO_OVERFLOW)” dentro de HashFile::find_by_id — ou seja, o tamanho da lista encadeada de overflow do bucket mais a leitura do bloco principal. O ID 5 caiu em um bucket cujo registro está no primeiro slot, retornando em 1 bloco; o ID 1483294 caiu em um bucket com ~49 blocos de overflow, custando 50 blocos em 4.096 B e 20 em 8.192 B, porque cada bloco de overflow agora comporta 5 registros em vez de 2, reduzindo o número de blocos encadeados. Já o seek1 executa sempre o mesmo percurso BPlusTree::find_leaf → find sobre a árvore 5 blocos com altura 3, ou 4 com altura 2, e em seguida lê 1 bloco de dado no arquivo paginado — por isso seus totais são praticamente constantes, independentemente do ID consultado. O contraste entre as duas colunas ilustra o ponto central do trabalho: o hashing tem custo

variável dependente da profundidade de overflow do bucket, enquanto a B+Tree tem custo previsível dado pela altura da árvore.

3.2 Ids Inexistentes

ID	findrec 4096	findrec 8192	seek1 4096	seek1 8192
1	45	18	4	3
2000000	41	17	4	3

Para IDs inexistentes, o findrec é obrigado a percorrer a lista de overflow inteira do bucket antes de concluir que o registro não existe, por isso os valores 45 e 41 são ainda maiores que o pior caso observado para IDs existentes em 4.096 B (50). Esse é o comportamento esperado: HashFile::find_by_id só retorna false quando block_idx chega a NO_OVERFLOW, ou seja, depois de visitar todos os blocos do bucket. Já o seek1 economiza 1 bloco, ficando com 4 em 4.096 B, ou 3 em 8.192 B, porque o código só executa a leitura do dado se found == true no índice: quando BPlusTree::find retorna falso, a linha hf.data_file().read_block(...) nunca é atingida. Isso mostra que uma busca negativa é mais barata em ambas as estruturas quando comparada à busca positiva correspondente.

3.4 Títulos

Consulta	findtitle 4096	findtitle 8192	seek2 4096	seek2 8192	hit
A rotating-magnet...	102.659	41.681	5+142 = 147	4+181 = 185	1
Review of particle physics...	102.659	41.681	5+53 = 58	4+254 = 258	45
Two-dimensional gas...	102.659	41.681	5+213 = 218	4+213 = 217	26
Um título que nao existe	102.659	41.681	4	3	0

findtitle sempre custa o total de blocos do arquivo menos o header global — HashFile::scan_by_title chama scan_all, que percorre for (i = 1; i < block_count; ++i), lendo cada bloco uma única vez. Como o hashing não preserva ordem, não há como parar cedo. Os valores são idênticos para todos os títulos porque o custo não depende de k (número de hits): mesmo um título inexistente força a varredura completa. Em contrapartida, o seek2 desce a B+Tree pelo fingerprint dos 4 primeiros bytes do título (4 ou 5 blocos, conforme a altura) e percorre as folhas encadeadas coletando todos os offsets com essa chave. Para cada offset retornado, seek2.cpp lê o bloco de dados e aplica o filtro final r.titulo == titulo, o que explica

por que o custo cresce com o número de falsos positivos e não apenas com o número de hits reais. No caso de 4.096 B, "A rotating-magnet..." Teve 142 leituras de dado para retornar 1 hit, indicando que existem ~141 títulos no dataset cujos 4 primeiros bytes coincidem com "A ro".

3.5 Intervalos

Intervalo	#hits	findrange 4096	findrange 8192	seekrange 4096	seekrange 8192
[1, 50]	10	102.659	102.65941.681	4+10 = 14	3+10 = 13
[1000, 1100]	20	102.659	41.681	4+20 = 24	3+20 = 23
[500000, 500500]	99	102.659	41.681	5+99 = 104	3+99 = 102
[200000, 260000]	3052	102.659	41.681	22+3052 = 3074	12+3052 = 3064
[1600000, 1700000]	0	102.659	41.681	4	3

findrange reusa HashFile::scan_by_range, que também chama scan_all — daí o custo constante igual ao total de blocos do arquivo. Não importa se o intervalo é vazio ([1600000, 1700000]), pequeno ([1, 50]) ou grande ([200000, 260000]): o hashing nunca permite podar avarredura, porque registros com IDs adjacentes caem em buckets arbitrários. Já o seekrange segue o caminho inverso: BPlusTree::range desce a árvore pelo limite inferior (3 a 5 blocos) e percorre next_leaf enquanto as chaves estiverem no intervalo, coletando a key e offset de cada registro — depois seekrange.cpp lê um bloco de dado por offset. Por isso o custo total decompõe-se em O(log n) (descida) + O(k) (leitura dos dados), o que explica diretamente a proporcionalidade observada: [1, 50] com 10 hits custa 4 + 10, [200000, 260000] com 3.052 hits custa 22 + 3052. O caso [1600000, 1700000] mostra que um intervalo vazio custa apenas os blocos de descida (4 em 4.096 B, 3 em 8.192 B), pois range retorna assim que encontra a primeira folha com chave maior que o limite superior — sem nenhuma leitura de dado.

4.Comparação e explicação das divergências

4.1 findrec vs seek1

O hashing foi muito mais caro que o previsto para IDs que caíram em buckets profundos. Cada bucket comporta apenas 2 registros no bloco principal; com ~100 registros por bucket em média, existem ~50 blocos de overflow encadeados para
percorrer antes de encontrar o ID.

A previsão para findrec era de 1 bloco e para seek1 4 blocos, o findrec variou de 1 a 50 blocos, já seek1 variou de 5 a 6 blocos. A B+Tree é mais previsível; o hashing só ganha quando o ID está no slot 0 do bucket principal.

4.2 findtitle vs seek2

Previsão:

findtitle ≈ 102.660

seek2 ≈ 3 + k.

Medido:

findtitle = 102.659 blocos em todas as execuções.

seek2 = 4 a 258 blocos.

4.3 findrange vs seekrange

Previsão:

Findrange = 102.660 sempre

seekrange = 4 + folhas + k.

Medido:

findrange = 102.659 em todas as execuções
seekrange variou de 4, intervalo vazio, a 3.074, intervalo com 3.052 registros. O custo é O(log n + k).

4.4 Divergências específicas

1. seek2 : "Design and implementation..."` retornou 0 hits mesmo com findtitle retornando 1.

Possível Causa: o título usado como argumento diferia do título armazenado, possíveis caracteres invisíveis ou variação de grafia.

2. Colisões no índice secundário: “Review of particle physics" teve 45 hits em findtitle mas apenas 15 em seek2 com 4.096 B. A chave compacta de 4 bytes "Revi" colide com outros títulos e o find só retorna registros com a mesma chave, não o título completo. Em 8.192 B o

comportamento mudou (45 hits) — detalhado em 7.6.

3. findrec mais lento que seek1 para 4 dos 5 IDs existentes. A B+Tree é ordem de magnitude mais rápida em buckets profundos.

5. Efeito do tamanho do bloco

6.1 Busca que não encontra nada

Operação	4.096 B	8.192 B
findrec ID 1	45	18
seek1 ID 1	4	3
findrec ID 2000000	41	17
seek1 ID 2000000	4	3
findtitle inexist	102.659	41.681
seek2 inexistente	4	| 3

seek1 para ID inexistente é mais barato que para ID existente, porque não lê o bloco de dados.
findrec para ID inexistente é muito caro se o bucket tiver overflow — é preciso percorrer toda a lista encadeada para descartar.
findtitle inexistente paga o custo da varredura completa.

seek2 inexistente é o mais barato de todos, só a descida.

6.2 Título repetido (índice secundário)

O título "Review of particle physics particle data group" aparece em 45 registros.

Tamanho	findtitle	seek2	Divergência
4.096 B |	45 hits	15 hits	30 hits perdidos
8.192 B	45 hits	45 hits	OK

A causa da divergência em 4.096 B deve-se ao índice secundário, porque ele usa apenas os 4 primeiros bytes do título como chave. Registros com títulos que compartilham os 4 primeiros bytes, "Revi..." Por exemplo, mas diferem no resto caem em folhas diferentes da

B+Tree. A consulta find retorna somente os registros cujo offset está na mesma folha que o ponto de descida.
Em 8.192 B o nó folha comporta 680 chaves, então mais registros com a mesma chave permanecem na mesma folha. Isso explica por que 8.192 B encontrou 45 hits e 4.096 B encontrou 15.

O índice secundário com chave compacta de 4 bytes tem recall dependente do tamanho do bloco. Uma solução mais robusta seria usar 8 bytes, ou o título completo como chave, ao custo de B+Trees mais profundas.

6.3 Intervalos com cardinalidades diferentes

Intervalo	hits	findrange	seekrange
[1600000, 1700000]	0	102.659	4
[1, 50]	10	102.659	14
[1000, 1100]	20	102.659	24
[500000, 500500]	99	102.659	104
[200000, 260000]	3052	102.659	3074

findrange é constante independente do intervalo.
seekrange cresce linearmente com k e tem um custo fixo de ~3 blocos de descida da B+Tree.
Para intervalos vazios, o ganho do índice é ~25.000×.

6.4 Concentração de registros em poucos buckets

Com 2.048 buckets e 2 slots/bloco, a ocupação média é de ~100 registros por bucket. Isso significa que praticamente todos os buckets têm overflow, tipicamente com ~50 blocos encadeados.
Consequência medida: findrec variou de 1 bloco, bucket com registro no slot 0, a 50 blocos, bucket com registro no fim da lista de overflow. Isso é o preço do hashing sem análise da distribuição de chaves. Uma alternativa não adotada foi usar um número de buckets próximo de 204.285 / 2 = 102.140, o que eliminaria overflows mas desperdiçaria ~400 MB em blocos vazios. A decisão por 2.048 buckets é um trade-off explícito.

6.5 Falsos positivos no índice secundário

Em 8.192 B, o seek2 do "A rotating-magnet..." leu 81 blocos de dados para retornar 1 registro. O find retornou vários offsets com a mesma chave de 4 bytes. O filtro final r.titulo == título, descartou os falsos positivos, mas cada offset descartado custou 1 bloco de

dados lido.
Em 4.096 B, o mesmo consulta leu 142 blocos. A diferença decorre de os registros estarem mais espalhados em blocos maiores.

7. Declaração de uso de ferramentas de IA

7.1 Ferramentas utilizadas

Claude, modelo Sonnet 4.5, em modo conversacional. Foi usado para discutir o layout do registro em disco, revisar o cálculo da ordem da B+Tree e a estrutura de nós, gerar esqueletos iniciais e ajustar assinaturas de funções à camada blockio fornecida pelo template, revisar a implementação do overflow do hashing, redigir este relatório, analisar os números medidos e produzir as tabelas comparativas.

7.2 O que foi solicitado

Explicações conceituais sobre hashing e B+Tree, geração de código C++ para HashFile, BPlusTree, upload e os seis programas de consulta, adaptados ao template do professor. Revisão dos números medidos e comparação com o previsto.

7.3 O que foi modificado após a geração

Ajustes nas assinaturas para casar com a camada blockio, a IA usou inicialmente uma interface diferente. Correção de dois erros de compilação: inclusão faltante de storage.hpp em hash_file.hpp e de <cstdint> em csv_reader.hpp. Escolha do número de buckets após considerar alternativas e todo o código entregue foi compilado e executado localmente em Ubuntu com Docker, produzindo os números reportados.